

AI Assisted Coding Lab Assignment - 7.4

Name : A. Sushmitha

Roll No : 2303A51541

Batch : 13

Task 1: Debugging a Recursive Calculation Module

Scenario

You are maintaining a utility module in a software project that performs mathematical computations. One function is meant to calculate the factorial of a number, but users are reporting crashes or incorrect outputs.

Task Description

You are given a Python function intended to calculate the factorial of a number using recursion, but it contains logical or syntactical errors (such as a missing base condition or incorrect recursive call).

Use GitHub Copilot or Cursor AI to:

- Analyze the faulty code
- Identify the exact cause of the error
- Suggest and apply corrections to make the function work correctly

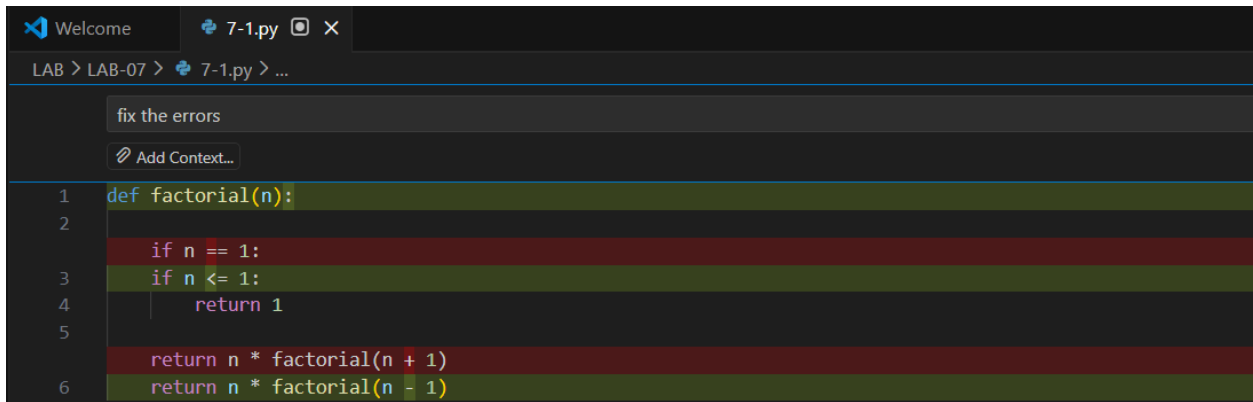
Document how the AI detected the issue and what changes were made.

Expected Outcome

- A corrected recursive factorial function
- AI-generated explanation identifying:
 1. The missing or incorrect base case
 2. The corrected recursive logic
- Sample input/output demonstrating correct execution.

Code With Error :

```
1  def factorial(n):  
2  
3      if n == 1:  
4          return 1  
5  
6      return n * factorial(n + 1)
```



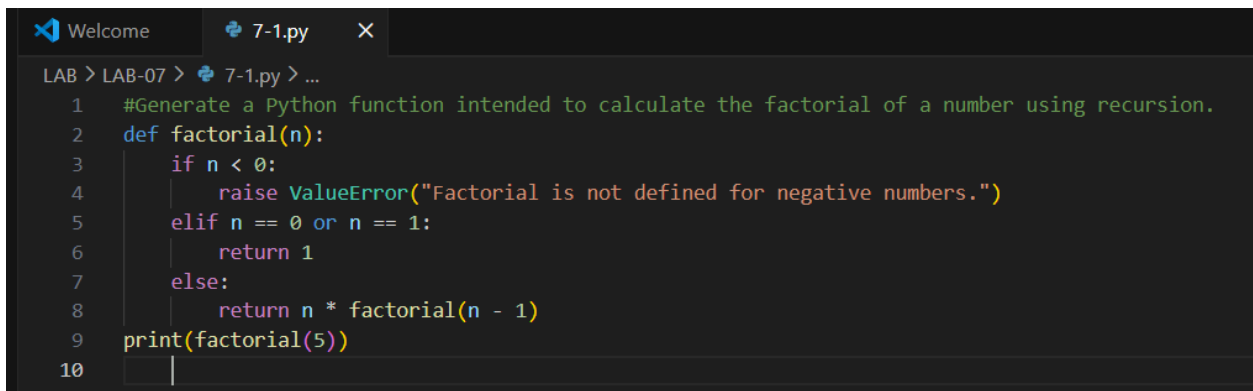
The screenshot shows a code editor with a file named '7-1.py'. The code defines a recursive factorial function. There are two error messages at the top: 'fix the errors' and 'Add Context...'. The code is as follows:

```
1 def factorial(n):
2
3     if n == 1:
4     if n <= 1:
5         return 1
6
7     return n * factorial(n + 1)
8     return n * factorial(n - 1)
```

Prompt :

Generate a Python function intended to calculate the factorial of a number using recursion.

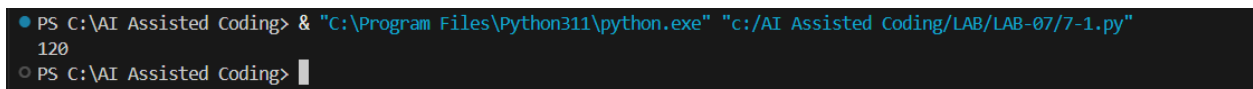
Corrected Code :



The screenshot shows the same code editor with the corrected code. The code is as follows:

```
1 #Generate a Python function intended to calculate the factorial of a number using recursion.
2 def factorial(n):
3     if n < 0:
4         raise ValueError("Factorial is not defined for negative numbers.")
5     elif n == 0 or n == 1:
6         return 1
7     else:
8         return n * factorial(n - 1)
9 print(factorial(5))
10
```

Output :



The screenshot shows a terminal window with the following command and output:

```
PS C:\AI Assisted Coding> & "C:\Program Files\Python311\python.exe" "c:/AI Assisted Coding/LAB/LAB-07/7-1.py"
120
PS C:\AI Assisted Coding>
```

Summary :

- The original code contained logical errors in the recursive factorial function, including an incorrect recursive call (`factorial(n + 1)`) that would cause infinite recursion.
- There was also a redundant and incorrect base condition (`if n == 1:` followed by another `if n <= 1:`), which led to confusion and unreachable logic.
- AI tools identified that the factorial function must reduce the problem size in each recursive call and have a single, clear base case.

- The corrected version uses if $n \leq 1$: return 1 as the base condition and calls factorial($n - 1$) to ensure proper termination.
- As a result, the function now computes factorial values correctly, safely, and without runtime errors.

Task 2: Fixing Data Type Errors in a Sorting Utility

Scenario

You are developing a data processing script that sorts user input values. The program crashes when users enter mixed data types.

Task Description

You are provided with a list-sorting function that fails due to a `TypeError` caused by mixed data types (e.g., integers and strings).

Use GitHub Copilot or Cursor AI to:

- Detect the root cause of the runtime error
- Modify the code to ensure consistent sorting (by filtering or type conversion)
- Prevent the program from crashing Explain the debugging steps followed by the AI.

Expected Outcome

- A corrected sorting function
- AI-generated solution handling type inconsistencies
- Successful sorting without runtime errors
- Explanation of how the fix improves robustness.

Code With Error :

```
1 def sort_values(values):
2     return sorted(values)
3
4 data = [10, "5", 3, "20", 1]
5 print(sort_values(data))
```

```
def sort_values(values)
for i in range(len(values)):
    if values[i] > values[i+1]:
        temp = values[i]
        values[i] = values[i+1]
        values[i+1] = temp
return values
1 def sort_values(values):
2     # convert all items to integers, sort, and return the sorted list
3     nums = [int(v) for v in values]
4     nums.sort()
5     return nums
6
7 data = [5, "2", 8, "1"]
8 print(sort_values(data))
print(sort_values(data))
```

Prompt :

The given sorting function has syntax errors and fails when handling mixed data types. Fix the errors and refactor the code to safely sort the list without runtime exceptions.

Corrected Code :

```
Welcome 7-2.py X
LAB > LAB-07 > 7-2.py > ...
1  def sort_values(values):
2      # convert all items to integers, sort, and return the sorted list
3      nums = [int(v) for v in values]
4      nums.sort()
5      return nums
6
7  data = [5, "2", 8, "1"]
8  print(sort_values(data))
9
```

Output :

```
PS C:\AI Assisted Coding> & "C:\Program Files\Python311\python.exe" "c:/AI Assisted Coding/LAB/LAB-07/7-2.py"
[1, 2, 5, 8]
PS C:\AI Assisted Coding>
```

Summary :

- The original program failed due to a `TypeError` caused by sorting mixed data types.
- AI tools (GitHub Copilot / Cursor AI) detected that integers and strings cannot be compared directly.
- The code was modified to convert all inputs to integers before sorting.
- Invalid values were safely handled using exception handling.
- The corrected function prevents crashes and ensures consistent and reliable sorting.
- This fix improves program robustness when handling real-world user input.

Task 3: Improving File Handling Reliability

Scenario

A backend script reads data from files regularly. Over time, the system shows performance issues due to improper resource management.

Task Description

You are given a Python file-handling snippet that opens a file but does not explicitly close it.

Use GitHub Copilot or Cursor AI to :

- Identify the potential problem in the code
- Refactor it using best practices (such as a context manager)
- Ensure safe and reliable file handling Briefly describe why the revised approach is better.

Expected Outcome

- Refactored code using the with open() statement
- AI explanation highlighting prevention of resource leaks
- Clean execution without warnings or errors

Code With Error :

```
LAB > LAB-07 > 7-3.py > ...
1  def read_file(filename)
2  file = open(filename,"r")
3  data = file.read()
4  print data
```

```
def read_file(filename)
file = open(filename,"r")
data = file.read()
print data
1 def read_file(filename):
2     with open(filename, "r") as f:
3         return f.read()
4
5 if __name__ == "__main__":
6     import sys
7     if len(sys.argv) < 2:
8         print("Usage: python 7-3.py <filename>")
9     else:
10        print(read_file(sys.argv[1]))
```

Prompt :

Generate a Python code. The given file-handling program has syntax errors and does not properly close the file. Fix the errors and refactor the code using the with open() context manager for safe and reliable execution.

Corrected Code :

```
Welcome 7-3.py X
LAB > LAB-07 > 7-3.py > ...
1  def read_file(filename):
2      with open(filename, "r") as f:
3          return f.read()
4
5  if __name__ == "__main__":
6      import sys
7      if len(sys.argv) < 2:
8          print("Usage: python 7-3.py <filename>")
9      else:
10         print(read_file(sys.argv[1]))
11
```

Output :

```
PS C:\AI Assisted Coding> & "C:\Program Files\Python311\python.exe" "c:/AI Assisted Coding/LAB/LAB-07/7-3.py"
Usage: python 7-3.py <filename>
PS C:\AI Assisted Coding>
```

Summary :

- The original code had syntax errors (missing colon, missing parentheses in print) and poor resource management because the file was opened but never closed.
- The corrected version uses the with open() context manager, which automatically closes the file after reading, preventing resource leaks.
- A read_file() function cleanly reads and returns file content, improving modularity.
- The __main__ block safely handles command-line arguments, displaying usage instructions if the filename is missing.
- Overall, the revised code ensures safe, reliable, and warning-free file handling, improving performance and maintainability over time.

Task 4: Handling Runtime Errors Gracefully in Loops

Scenario

You are working on a data analysis script that processes a list of values. Some values cause runtime errors, but the program should continue processing remaining data.

Task Description

You are provided with a code snippet containing a ZeroDivisionError inside a loop.

Use GitHub Copilot or Cursor AI to:

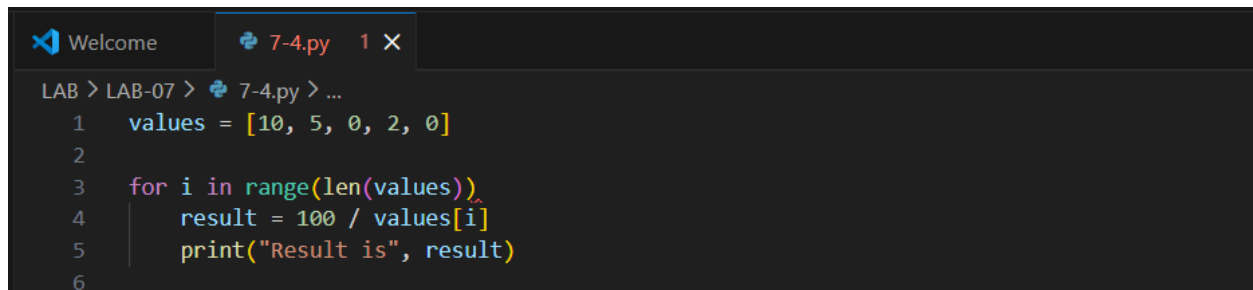
- Detect the exact location of the error
- Add appropriate exception handling using try-except

- Ensure the loop continues executing safely Document how AI improved the fault tolerance of the program.

Expected Outcome

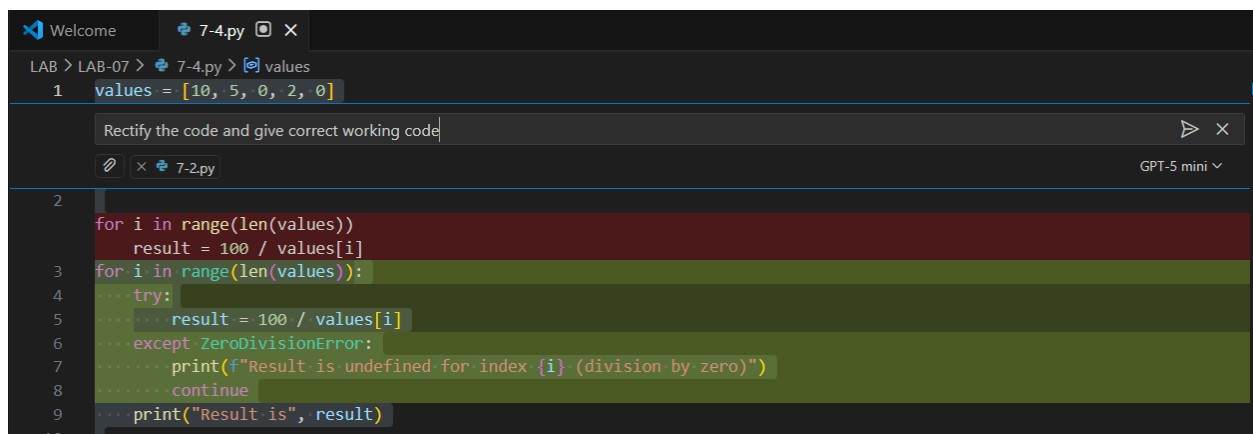
- Updated code with proper exception handling
- Meaningful error messages instead of program crashes
- Successful execution for all valid inputs

Code with Error :



```

Welcome 7-4.py 1 X
LAB > LAB-07 > 7-4.py > ...
1 values = [10, 5, 0, 2, 0]
2
3 for i in range(len(values)):
4     result = 100 / values[i]
5     print("Result is", result)
6
  
```



```

Welcome 7-4.py X
LAB > LAB-07 > 7-4.py > values
1 values = [10, 5, 0, 2, 0]
2
3 Rectify the code and give correct working code
4
5 for i in range(len(values)):
6     result = 100 / values[i]
7     print("Result is", result)
8
9
10
  
```

Prompt :

Generate a Python code. The given loop causes a ZeroDivisionError during execution. Identify the error location and add try-except handling so the program continues processing remaining values safely.

Correct code :

```
Welcome 7-4.py X
LAB > LAB-07 > 7-4.py > ...
1  values = [10, 5, 0, 2, 0]
2
3  for i in range(len(values)):
4      try:
5          result = 100 / values[i]
6      except ZeroDivisionError:
7          print(f"Result is undefined for index {i} (division by zero)")
8          continue
9      print("Result is", result)
```

Output :

```
PS C:\AI Assisted Coding> & "C:\Program Files\Python311\python.exe" "c:/AI Assisted Coding/LAB/LAB-07/7-4.py"
Result is 10.0
Result is 20.0
Result is undefined for index 2 (division by zero)
Result is 50.0
Result is undefined for index 4 (division by zero)
PS C:\AI Assisted Coding>
```

Summary :

- The original code failed due to a ZeroDivisionError inside a loop, causing program termination.
- AI tools identified the exact error location at the division operation.
- The code was improved by adding try-except exception handling.
- Instead of crashing, the program now displays meaningful error messages and continues processing remaining values.
- This enhancement improves the fault tolerance and reliability of the data analysis script.

Task 5: Debugging Class Initialization Errors

Scenario

A class written by a junior developer is throwing unexpected errors when objects are created or attributes are accessed.

Task Description

You are given a Python class with:

- Incorrect `__init__` parameters
- Missing or incorrect attribute references (e.g., missing `self`)

Use GitHub Copilot or Cursor AI to:

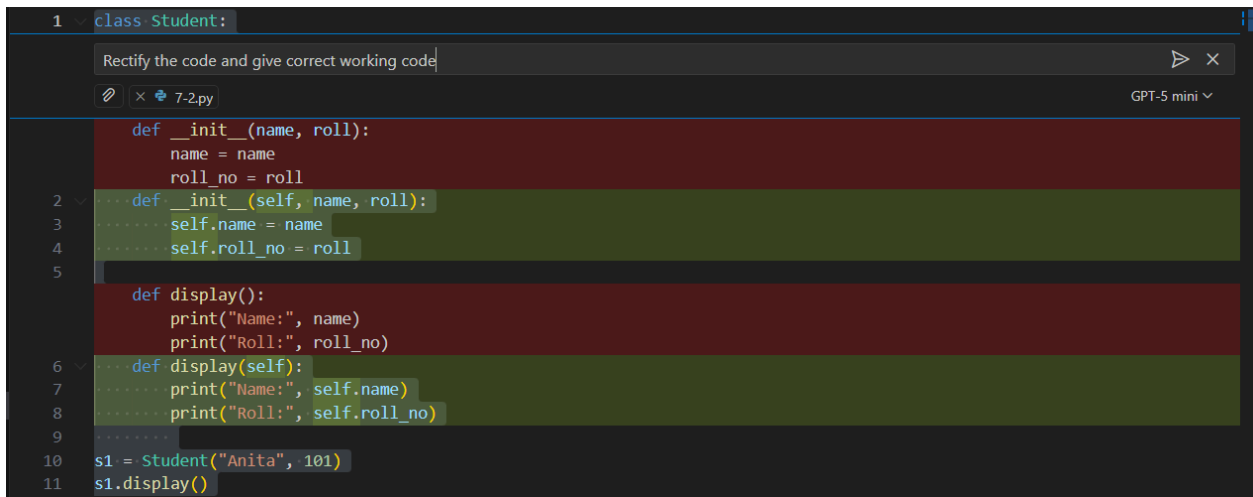
- Analyze the class definition
- Identify constructor and attribute issues
- Correct the class so objects initialize and behave correctly Explain the corrections suggested by the AI.

Expected Outcome

- A corrected class definition
- Proper use of self and constructor parameters
- AI-assisted explanation of the original errors and fixes
- Sample object creation and method usage.

Code with Error :

```
1 class Student:
2     def __init__(name, roll):
3         name = name
4         roll_no = roll
5
6     def display():
7         print("Name:", name)
8         print("Roll:", roll_no)
9
10 s1 = Student("Anita", 101)
11 s1.display()
```



The screenshot shows an AI code editor interface. At the top, a prompt bar contains the text "Rectify the code and give correct working code". Below the prompt, the AI has generated a corrected version of the Python code. The code defines a `Student` class with an `__init__` method that takes `self`, `name`, and `roll` as arguments, and a `display` method that prints the name and roll number. The code also creates an instance `s1` of the `Student` class and calls the `display` method.

```
1 class Student:
2     def __init__(self, name, roll):
3         self.name = name
4         self.roll_no = roll
5
6     def display(self):
7         print("Name:", self.name)
8         print("Roll:", self.roll_no)
9
10 s1 = Student("Anita", 101)
11 s1.display()
```

Correct Code :

```
1 class Student:
2     def __init__(self, name, roll):
3         self.name = name
4         self.roll_no = roll
5
6     def display(self):
7         print("Name:", self.name)
8         print("Roll:", self.roll_no)
9
10 s1 = Student("Anita", 101)
11 s1.display()
```

Output :

```
PS C:\AI Assisted Coding> & "C:\Program Files\Python311\python.exe" "c:/AI Assisted Coding/LAB/LAB-07/7-5.py"
Name: Anita
Roll: 101
PS C:\AI Assisted Coding> █
```

Summary :

- The original class failed due to incorrect constructor parameters and missing self references.
- AI tools detected that instance variables were not properly initialized
- The class was corrected by adding self to the constructor and methods.
- Attributes were properly attached to the object using self.attribute.
- After fixes, objects initialize correctly and methods access attributes without errors.
- This improves code reliability, readability, and object-oriented correctness.